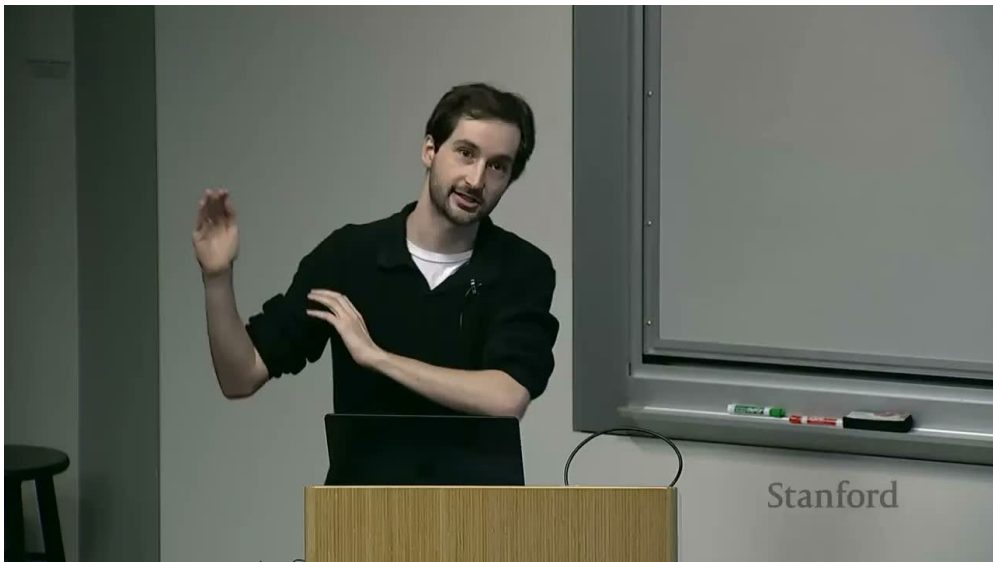


课程笔记

CS231n Lecture 7: Recurrent Neural Networks

Codex Harness

2026-04-15



视频作者/频道: Stanford Online / CS231n

发布日期: 2025-04-22

视频时长: 01:11:43

视频链接: <https://www.youtube.com/watch?v=kG2lAPBF7zA>

目录

1 为什么序列建模需要新的网络结构	2
1.1 本章小结	2
2 Vanilla RNN 的状态、参数共享与计算图	2
2.1 本章小结	4
3 BPTT、截断训练与梯度问题	4
3.1 本章小结	5
4 门控 RNN: LSTM 与 GRU 的核心思想	5
4.1 本章小结	7
5 语言模型、图像描述与序列到序列	7
5.1 本章小结	8
6 总结与延伸	8

1 为什么序列建模需要新的网络结构

在前几讲中，视觉模型大多假设输入是固定维度的向量、网格或特征图；网络读入一次输入，输出一次预测。但本讲一开始就明确指出，计算机视觉中的很多任务天然是序列（sequence）问题，而不是单次静态映射。典型例子包括：

- 图像描述（image captioning）：输入一张图像，输出一个单词序列。
- 动作预测（action prediction）：输入一串视频帧，输出动作类别。
- 视频描述（video captioning）：输入视频帧序列，输出一句话。
- 帧级别视频分类：输入随时间变化的特征流，输出每一步或整体标签。

这些任务的共同点是：模型不只关心“当前输入是什么”，还必须记住“之前已经看到了什么”。因此，课程把 **内部状态（internal state）** 作为序列模型的核心思想。换句话说，序列模型不是把整条序列一次性拍平成一个向量，而是在时间维度上持续维护一个可以被更新的记忆。

本讲主线

本讲不是简单地给出几个新层，而是沿着一条非常清晰的路径推进：先说明序列任务的需求，再引出 vanilla RNN 的参数共享与 hidden state，随后讨论 BPTT 的训练困难，最后用 LSTM/GRU 这样的门控结构改善长程依赖建模。

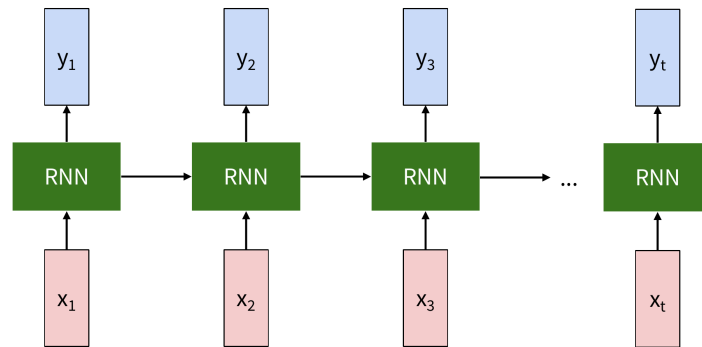
1.1 本章小结

序列任务与静态分类任务的差异，不在于输出形式是否更长，而在于模型必须在不同时间步之间持续传递信息。本讲后续所有结构，都是围绕“怎样保存和更新这份状态”展开的。

2 Vanilla RNN 的状态、参数共享与计算图

课程首先用“展开后的时间轴”来解释 RNN。图中每个时间步都会接收当前输入 x_t ，并从前一时刻继承 hidden state h_{t-1} ，再产生新的 hidden state h_t 与输出 y_t 。

Unrolled RNN



Stanford CS231n 10th Anniversary

Lecture 7 - 23

April 22, 2025

图 1: 将 RNN 沿时间展开后, 可以直观看到输入、状态与输出之间的依赖关系。

最核心的两条公式是:

$$h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

其中:

- x_t : 第 t 个时间步的输入向量。
- h_t : 第 t 个时间步的隐藏状态 (hidden state)。
- W_{xh} : 输入到隐藏状态的参数矩阵。
- W_{hh} : 前一时刻状态到当前状态的递归参数矩阵。
- W_{hy} : 隐藏状态到输出的映射矩阵。
- $\phi(\cdot)$: 逐元素非线性函数, 课程里常用 $\tanh$ 或 ReLU 。

RNN 与普通前馈网络最关键的差别

同一组参数 W_{xh}, W_{hh}, W_{hy} 会在所有时间步重复使用。这就是所谓的 **参数共享 (parameter sharing)**。它让模型可以处理任意长度序列, 同时把“时间结构”编码进网络里。

为了避免公式看起来过于抽象, 课程还给了一个很有代表性的玩具例子: 在二值序列里检测“重复的 1”。这个例子的重点不是任务本身, 而是让读者思考: **hidden state 到底应该记什么?** 结论很直接, 若想判断当前是否又看到了一个 1, 状态里至少要编码“上一个输入是不是 1”。这说明 hidden state 不是任意的缓存, 而是任务相关信息的压缩表示。

接着, 课程把 RNN 计算图分成几种常见模式:

- **many-to-many**: 输入与输出都是序列, 例如逐帧标注。

- **many-to-one**: 输入是序列，输出是单标签，例如情感分类或动作识别。
- **one-to-many**: 输入是单个向量，输出是序列，例如图像描述。

这种分类非常重要，因为它直接决定损失函数在哪些时间步上定义、反向传播怎样展开，以及推理时需要保留哪些中间状态。

2.1 本章小结

Vanilla RNN 用一条反复复用的递归公式，把“当前输入”和“历史状态”融合成新的状态。参数共享赋予它处理变长序列的能力，而 hidden state 则承担了跨时间步的信息压缩任务。

3 BPTT、截断训练与梯度问题

有了前向递归之后，训练就自然落到 **Backpropagation Through Time, BPTT**。本质上，它并不是发明了新的反向传播，而是先把 RNN 在时间维上展开成一个很深的网络，再对这个展开图做标准反传。

如果把每个时间步的损失记作 L_t ，那么对状态的梯度递推可以写成：

$$\frac{\partial L}{\partial h_t} = \frac{\partial L_t}{\partial h_t} + \frac{\partial L}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_t}$$

继续展开后，会出现一串 Jacobian 连乘：

$$\frac{\partial L}{\partial h_t} \propto \prod_{k=t+1}^T \frac{\partial h_k}{\partial h_{k-1}}$$

这也是课程后来反复强调 **vanishing / exploding gradients** 的根源。如果这串矩阵的有效奇异值长期小于 1，梯度会指数级衰减；如果长期大于 1，梯度又会爆炸。RNN 越想记住更久远的信息，链就越长，问题就越严重。

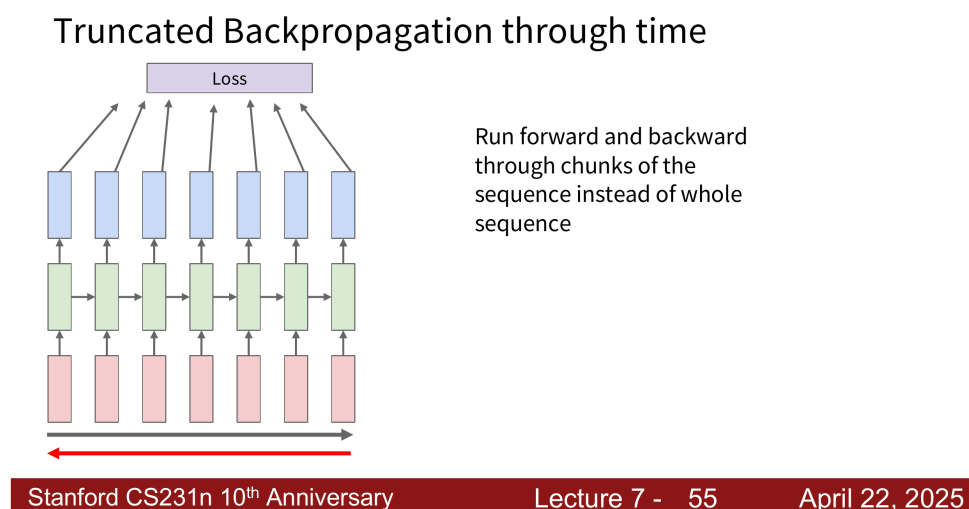


图 2: Truncated BPTT 用固定窗口回传梯度，是训练长序列时最重要的工程折中之一。

因此，课程给出的第一个工程化应对手段不是“换掉 RNN”，而是 **truncated BPTT**：前向仍然可以持续滚动更长的序列，但反向只在一个有限窗口内展开。这样做的收益很直接：

- 显著降低显存与计算开销。
- 让 mini-batch 训练更容易落地。
- 把优化问题从“无限长依赖”改成“局部窗口中的近似优化”。

代价也同样明确：如果真正重要的依赖跨度超出截断窗口，模型就很难从梯度里得到足够信号。

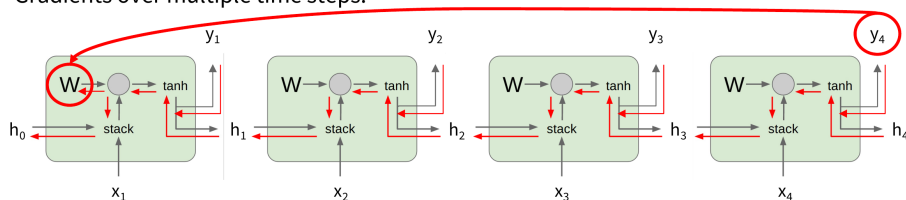
不要误解 truncated BPTT

截断不是理论上更正确的训练方式，而是工程上可承受的近似。它解决的是训练可行性，不是表达能力。

课程后半部分还专门用梯度流图指出：vanilla RNN 即便在前向上能“理论上”记住长信息，反向优化也常常先失败。也就是说，难点不只是模型有没有记忆槽位，而是优化器是否真的能把有用信息写进这些槽位。

Vanilla RNN Gradient Flow

Gradients over multiple time steps:



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W} \quad \text{What if we assumed no non-linearity?}$$

图 3: 沿时间展开的梯度流展示了 recurrent Jacobian 连乘为何会让长依赖训练变得脆弱。

3.1 本章小结

BPTT 把 RNN 训练问题变成了一个很深的展开网络训练问题。真正的瓶颈不在“能不能写出递推公式”，而在于梯度沿时间传播时是否还能保持稳定。截断 BPTT 是实用近似，梯度消失/爆炸是核心优化障碍。

4 门控 RNN: LSTM 与 GRU 的核心思想

既然问题出在“所有信息都挤在同一个 hidden state 里”，课程自然引出更复杂的门控结构。LSTM 的核心改动，是把“短时非线性状态”和“更稳定的 cell state”区分开来，再通过若干门

(gates) 决定信息如何写入、保留和读出。

一种标准写法如下：

$$\begin{aligned}f_t &= \sigma(W_f[x_t; h_{t-1}] + b_f), & i_t &= \sigma(W_i[x_t; h_{t-1}] + b_i) \\ \tilde{c}_t &= \tanh(W_c[x_t; h_{t-1}] + b_c), & o_t &= \sigma(W_o[x_t; h_{t-1}] + b_o) \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, & h_t &= o_t \odot \tanh(c_t)\end{aligned}$$

其中：

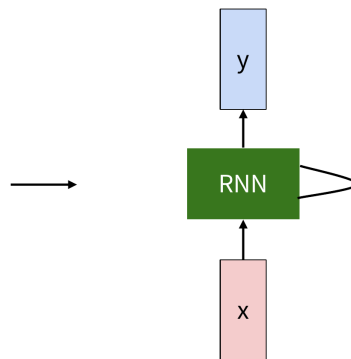
- f_t : forget gate, 决定旧记忆保留多少。
- i_t : input gate, 决定新信息写入多少。
- $\tilde{c}_t$: candidate memory, 表示候选写入内容。
- o_t : output gate, 决定当前暴露多少内部记忆作为输出。
- c_t : cell state, 是跨时间传递的“主记忆通道”。
- $\odot$: 逐元素乘法。

THE SONNETS

by William Shakespeare

From fairest creatures we desire increase,
That thereby beauty's rose might never die,
But as the ripper should by time decrease,
His tender heir might bear his memory;
But thou, contracted to thine own bright eyes,
Feed'st thy light's flame with self-substantial fuel,
Making a famine where abundance lies,
Thyself thy foe, to thy sweet self too cruel:
Thou that art now the world's fresh ornament,
And only herald to the gaudy spring,
Within thine own bud burdest thy content,
And render chart mak'st waste in niggarding:
Pity the world, or else this glutton be,
To eat the world's due, by the grave and thee.

When forty winters shall besiege thy brow,
And dig deep trenches in thy beauty's field,
Thy youth's proud livery so gazed on now,
Will be a tatter'd weed of small worth held:
Then being asked, where all thy beauty lies,
Where all the treasure of thy lusty days;
To say, within thine own deep sunken eyes,
Were an all-eating shame, and thriftless praise.
How much more praise-deserv'd thy beauty's use,
If thou couldst answer 'This fair child of mine
Shall sum my count, and make my old excuse,'
Proving his beauty by succession thine!
This were to be new made when thou art old,
And see thy blood warm when thou feel'st it cold.



[Blogpost from Andrej Karpathy back in 2015!](#)

Stanford CS231n 10th Anniversary

Lecture 7 - 68

April 22, 2025

图 4: LSTM 通过 forget/input/output 三类门，把“保留什么、写入什么、暴露什么”显式分开。

从教学角度看，LSTM 真正解决的不是“让梯度永不衰减”，而是给网络提供一条更容易保留长期信息的通路。因为 c_t 中包含了 $f_t \odot c_{t-1}$ 这一相对直接的残差式传播路径，只要 forget gate 不把旧状态完全抹掉，跨时间的信息保存就比 vanilla RNN 容易得多。

课程也特别提醒：**LSTM 并不神奇**。它只是让长依赖更可训练，而不是在数学上彻底消灭消失/爆炸梯度。模型仍然可能因为门值设置不当、初始化不佳或序列太长而训练困难。

在此基础上，GRU (Gated Recurrent Unit) 可以理解为更紧凑的门控变体：它把状态更新路径进一步简化，用更少的门与更少参数换来更轻量的实现。课程的重点仍放在 LSTM，但把 GRU 摆在同一脉络里，是为了强调“门控”才是核心思想，而不是某一个特定公式版本。

4.1 本章小结

LSTM/GRU 的本质不是把 RNN 变复杂，而是把状态更新从单次非线性变换改造成“受控的信息流”。一旦把长期记忆通道和即时输出通道拆开，模型就更有机会同时保留旧信息、吸收新信息并稳定训练。

5 语言模型、图像描述与序列到序列

本讲最后把 RNN 拉回到两个最典型的应用：语言建模 (language modeling) 与图像描述 (image captioning)。

在语言模型里，训练目标通常是：给定前缀 $w_1, \dots, w_{t-1}$ ，预测下一个 token w_t 。如果把前一个词的 embedding 作为输入，RNN 每一步都会更新状态并输出一个词表分布。训练时常用 teacher forcing，也就是把真实前缀喂给模型；推理时则改为把模型自己刚生成的 token 再送回去。

```
1 h = h0
2 loss = 0
3 for t in range(T):
4     x_t = embed(tokens[t])
5     h = rnn_cell(x_t, h)
6     logits = W_hy @ h + b_y
7     loss += cross_entropy(logits, tokens[t + 1])
```

Listing 1: RNN decoder / language model 的典型 teacher forcing 训练流程

图像描述则是在这个框架前面再接一个视觉编码器。最常见的思路是：

- 用 CNN 或其他视觉 backbone 把图像编码成向量或空间特征。
- 用这个视觉表征初始化 decoder 的 hidden state，或者把它作为每步附加输入。
- 让 RNN 逐词生成描述。

Image Captioning: Example Results

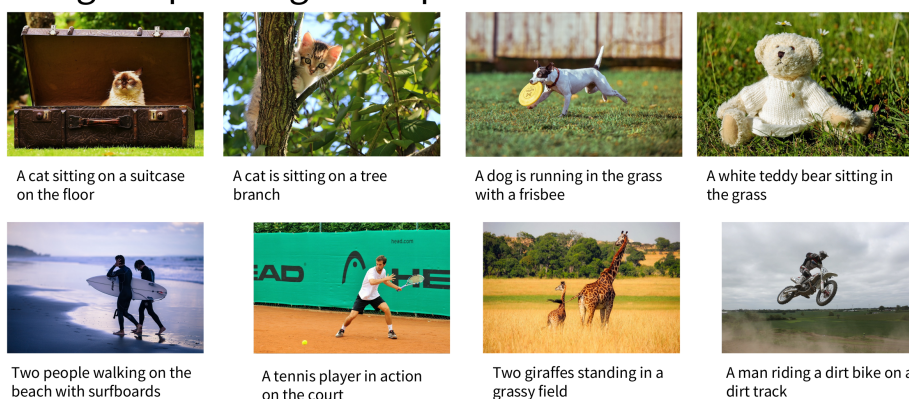


图 5: 图像描述把视觉表征和词序列生成结合起来，是 RNN 在视觉任务中的经典应用。

这类方法的优点是结构直观、训练范式清晰，也天然适合 one-to-many 与 sequence-to-sequence 任务。但它也暴露出前面说过的瓶颈：所有历史上下文都被压进单条状态链中，越长的描述、越复杂的依赖、越难优化。

为什么下一讲会转向 attention

当句子很长、输入很长，或 decoder 在某个时间步只需要关注输入中的局部区域时，单个 hidden state 往往成为瓶颈。attention 的价值，就是允许模型在每一步“直接回看”更丰富的上下文，而不必把一切都塞进同一个状态向量。

5.1 本章小结

RNN 的应用逻辑非常统一：先维护状态，再按任务把状态读成分类、序列或文本。语言模型和图像描述说明了它的表达力，也把 hidden-state bottleneck 的问题暴露得最清楚。

6 总结与延伸

本讲把序列建模的基本问题讲得非常系统：为什么序列需要内部状态、vanilla RNN 如何通过参数共享处理变长输入、BPTT 为什么训练困难、LSTM/GRU 如何通过门控改善长依赖，以及这些结构如何落到语言模型与图像描述里。

从今天回看，RNN 的历史意义并不只在于它曾经主导序列任务，更在于它把几个后来一直延续的问题摆到了台面上：

- 状态应该如何压缩历史信息？
- 长程依赖为什么难训练？
- 计算图应该怎样在表达力、并行性与优化稳定性之间折中？

课程在结尾已经给出下一步方向：一方面，像 LSTM、GRU、modern state-space models（例如 Mamba）这样的结构继续探索更好的状态更新；另一方面，attention 则从根本上放宽了“所有历史都必须通过单一状态链传递”的限制。下一讲的核心，就是这条新路线如何出现并最终演化成 Transformer。